CS3230 : Design and Analysis of Algorithms (Spring 2014)
Tutorial Set #4
[SOLUTION SKETCHES]

R1. (Exercise 8.2-1 on p. 196 of [CLRS]) [Exercising Counting-Sort]
Using Figure 8.2 (or lecture note diagram) as a model, illustrate the operation of COUNTING-SORT on the array $A = [6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2]$.

SOLUTION: Please check back after HW2 is due. ☺

R3. [Finding Min-and-Max simultaneosly]
This problem deals with finding both the *minimum and maximum* of n elements using as few comparisons as possible. One simple way is to just call Find-Min and Find-Max separately, which will take $2(n-1)$ comparisons. But, we can do better.
Given an algorithm to find both the *minimum and maximum* of n elements using as most $3\lceil n/2 \rceil$ comparisons (in the worst case).

SOLUTION: Read [CLRS]-C9.1.]

D1. [Tri-Partitioning for a better Quicksort?]
In *standard* Quicksort, we partition an array $A[1..n]$ about a pivot element x. Prof. Triple suggests that we might be able to do better if we use *two* pivot elements, instead of just one. (He thinks that a good *tripartite partition* works better, just like the *good tripartite industrial relations* in Singapore.)

(a) Suppose that you choose *two* elements x and y (for simplicity, assume that $x < y$) to partition the array $A[1..n]$ into three partitions: the left partition with all elements $\leq x$, the middle partition with all elements $\geq x$ and $\leq y$, and the right partition with all elements $\geq y$. Give an algorithm to perform this tri-partitioning with two pivots.
[*Hint:* Extend/Generalize the partitioning algorithm given in the lectures (L04b-S4) to do partitioning into three parts, small elements, medium elements, large elements.]

SOLUTION: Here, you are generalizing the bi-partitioning algorithm to tri-partitioning. So, you can reuse the bi-partitioning algorithm. Recall, that you can (i) re-use the result, or (ii) you can re-use the method.

(i) **Re-using the result:** No need new partitioning algorithm. Use the bigger pivot y to do a bi-partitioning on y into left-partition containing the small-and-medium elements, and right-partition containing the large elements. Then, call algorithm again to do a partitioning of the "left-partition" using pivot x to separate the small and the medium elements. *Viola*!

(ii) **Re-using the method:** Here we look at the idea behind the algorithm and try to generalize the algorithmic idea. Instead of maintain two partitions, we now maintain three partitions. Let late Prof. Dijkstra coined the name the Dutch National Flag problem for this and also gave an elegant algorithm. Steven Halim provided a web-link at
http://en.wikipedia.org/wiki/Dutch_national_flag_problem

(b) What is the best-case for tri-partitioning. In that case, give the corresponding recurrence and solve it to get the best-case running time.

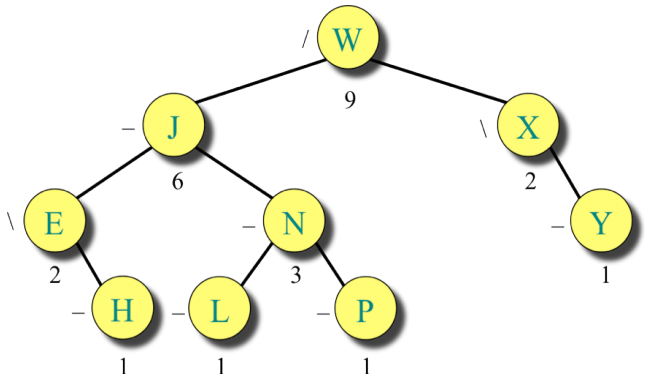
SOLUTION: (from Steven Halim)
The best-case happens when all the 3 partitions are of size $n/3$, giving
$T(n) = 3*T(n/3) + \Theta(n)$

$f(n) = \Theta(n)$, $a=3$, $b=3$. So $n^{\log_b a} = n^{\log_3 3} = n^1$.
Since $f(n) = \Theta(n) = \Theta(n^{\log_b a})$. This is **Case 2**. Hence, $T(n) = \Theta(n \lg n)$
Thus, the best case for tri-partitioning is still $\Theta(n \lg n)$, the same as for bi-partitioning.

Discussion: So, is there a merit of doing tri-partition vs bi-partition?
Assuming that we are sorting integers, we can set $y = (x+1)$ where x is a random pivot, so that the Quicksort can "ignore" items that *are equal to* the pivot x – don't recursively sort the middle partition. This can be slightly faster if there are many repeated elements.

D2. [Augmenting DS: An OS Tree made from an AVL Tree]
Consider the following binary search tree T, with the *keys* shown inside the node. The tree T is also an AVL-tree with the "*balance*" information { /, –, \ } indicated next to the node. The *balance* information for the top three nodes are filled in already.

SOLUTION SKETCH: (a), (b)



(a) Your first task is to fill in the balance information for all the other nodes in the tree.
[If you have not learnt AVL trees before, read the uploaded notes on "AVL trees".]

(b) Next, show how to augment the AVL-tree T into an OS Tree with the size information (as defined in Lectures on "Augmenting Data Structures". Show the size information below each node in the tree.

(c) Show how the call OS-SELECT (T.root, 4) operates on the augmented AVL-tree T.

SOLUTION SKETCH: (From Steven Halim)

Visit node W;	W has rank 6+1 = 7,	Go left & Find rank 4 element.
Visit node J;	J has rank 2+1 = 3,	Go right & Find rank 4-3 = 1 element. (smallest)
Visit node N;	N is rank 1+1 = 2,	Go left & Find rank 1 element.
Visit node L;	L is rank 0+1 = 1,	Done, found the answer L.

(d) Show how the call OS-RANK (*T*.root, Y) operates on the augmented AVL-tree *T*.

SOLUTION SKETCH: (From Steven Halim)

Start at node Y;	init $r = 0+1 = 1$;	Move to parent node X;
Now at node X;	$r = 1 + (0+1) = 2$;	Move to parent node W
Now at node W;	$r = 2 + (6+1) = 9$;	This is the root. Done.

Rank of node Y is 9.

EXERCISE: (from LeongHW) Now do OS-RANK (*T*.root, L)

(e) Show how the call OS-INSERT (*T*.root, I) operates on the augmented AVL-tree *T*.

SOLUTION SKETCH: (Follow Steven's solution. Will clean this part up later.)

e. W>I, go left
J>I, go left again
E<I, go right
H<I, go right
insert I as the right subtree of H, unbalance at E, rotate left at E

The new AVL tree looks like this:

```
          W
        /   \
       J     X
      / \     \
     H   N     Y
    / \ / \
   E  I L  P
```

The subtree sizes are updated accordingly: E=I=1, H=2, J=7, W=10 now.

(f) Show how the call OS-DELETE (*T*.root, Y) operates on the *original* augmented AVL-tree *T*.

SOLUTION SKETCH: (Follow Steven's solution. Will clean this part up later.)

f. W<Y, go right
X<Y, go right
Y is a leaf vertex, just delete it
but it will cause imbalance at W, we do rotate left at J and rotate right at W

The new AVL tree looks like this:

```
          N
        /   \
       J     W
      / \   / \
     E   L P   X
      \
      H
```

The subtree sizes are updated accordingly: X=1, W=3, N=8, J=4 now.

D3. Exercise 9.1-1 (p.215)
Second largest of n elements in $(n + \lceil \lg n \rceil - 2)$ comparisons in the worst case.

SOLUTION SKETCH: Build a balanced-tournament on n elements, using $(n-1)$ comparisons. Height of tournament is $\lceil \lg n \rceil$. Then 2nd largest must be "a loser" on the path of the champion. There are only $\lceil \lg n \rceil$ elements in this path. So, we need only $(\lceil \lg n \rceil - 1)$ comparisons. Total is $(n + \lceil \lg n \rceil - 2)$ comparisons in the worst-case.

D4. [Alternative Decision Tree]
In the decision tree (for sorting 3 elements) shown in the lectures (Slide L05-S5.4]), we compare 1:2 (namely, a_1:a_2) at the root of the tree.

Give an alternative decision tree that compares 2:3 (namely, a_2:a_3) at the root node of the tree and also have *height* 3 (meaning that the worst-case time to sort is still 3 comparisons).

SOLUTION SKETCH: (Follow Steven's solution. Will clean this part up later.)

4. the easiest answer to generate the decision tree:
from lecture note (or CLRS chapter 8) example that starts with 1:2,
replace all 1 with 2, and replace all 2 with 3, and 3 with 1

we have:

```
                    2:3
          /                       \
       3:1                            2:1
     /    \                        /      \
<2,3,1>    2:1              <3,2,1>      3:1
          /     \                       /      \
     <2,1,3> <1,2,3>               <3,1,2> <1,3,2>
```